k-NN Search: From Linear Scan to KD-tree

Wook-Shin Han

Spring 2026

Motivation & Setup (with Metric Definition)

- **k-NN** (k-Nearest Neighbors): For a query $\mathbf{q} \in \mathbb{R}^d$, find the k points in $\mathcal{X} = \{\mathbf{x}_i\}_{i=1}^n$ closest to $\mathbf{q}$ under a *distance function* $d(\cdot, \cdot)$.
- **Metric (formal):** A pair $(\mathcal{X}, d)$ is a *metric space* if for all $x, y, z \in \mathcal{X}$,

$$\begin{aligned}
&\text{(Non-negativity)} && d(x, y) \geq 0, \\
&\text{(Identity)} && d(x, y) = 0 \iff x = y, \\
&\text{(Symmetry)} && d(x, y) = d(y, x), \\
&\text{(Triangle)} && d(x, z) \leq d(x, y) + d(y, z).
\end{aligned}$$

- **Why this matters:** Metric properties—especially the triangle inequality—enable *sound pruning* in tree indexes (e.g., KD-tree/ball-tree) via valid lower bounds.
- **Principled path:** Start from the *no-index* linear scan (exact, simple, clear bound), then accelerate with data structures that exploit metric properties to prune unlikely regions.
- In high dimensions, pruning often weakens (*curse of dimensionality*); the **baseline** remains a strong comparator.

Baseline (No Index): Linear Scan + Size-k Max-Heap

- **Idea:** Compare $\boldsymbol{q}$ to every point once; keep current k best in a **fixed-size max-heap**.
- **Exactness:** *Exact k-NN* without any index.
- **Complexity:** Each heap update costs $O(\log k) \Rightarrow$ total $\boxed{O(n \log k)}$. With distance computations: $O(nd + n \log k)$.
- **Pros:** Simple, cache-friendly; very competitive when d is high or n is small/medium.

Pseudocode: k-NN by Linear Scan (No KD-tree)

```
# X: list/array of points; q: query point; k: number of neighbors
# dist() returns squared distance to avoid sqrt

def knn_linear_scan(X, q, k):
    PQ_nbrs = MaxHeap(capacity=k)  # key = dist(q, point); top = current worst
    tau = float('inf')

    for p in X:                    # single pass over all n points
        d = dist(q, p)
        if len(PQ_nbrs) < k:
            PQ_nbrs.push((d, p))
            tau = PQ_nbrs.max_key()
        elif d < PQ_nbrs.max_key():
            PQ_nbrs.replace_max((d, p))
            tau = PQ_nbrs.max_key()

    return sorted(PQ_nbrs.items(), key=lambda t: t[0])  # ascending by distance
```

Notes: Use squared distances; standardize features to balance scales; vectorize distance computation for CPU/GPU efficiency.

Using k-NN for Prediction

Classification: majority vote among neighbors $\mathcal{N}_k(\boldsymbol{q})$.

$$\hat{y}(\boldsymbol{q}) = \arg\max_{c \in \mathcal{C}} \sum_{(\boldsymbol{x}_i, y_i) \in \mathcal{N}_k(\boldsymbol{q})} \mathbf{1}[y_i = c]\, w_i, \quad w_i = \begin{cases} 1 & \text{(uniform)} \\ \dfrac{1}{\mathrm{d}(\boldsymbol{q}, \boldsymbol{x}_i) + \varepsilon} & \text{(distance)} \\ \dfrac{1}{\left(\mathrm{d}(\boldsymbol{q}, \boldsymbol{x}_i) + \varepsilon\right)^p} & \text{(inverse-power)} \end{cases}$$

Regression: weighted average over neighbors.

$$\hat{f}(\boldsymbol{q}) = \frac{\sum_{(\boldsymbol{x}_i, y_i) \in \mathcal{N}_k(\boldsymbol{q})} w_i\, y_i}{\sum_{(\boldsymbol{x}_i, y_i) \in \mathcal{N}_k(\boldsymbol{q})} w_i}.$$

Notes: In scikit-learn, set `weights='uniform'` or `weights='distance'`; for the inverse-power form, pass a callable. If any neighbor has zero distance, scikit-learn predicts using only the zero-distance neighbors. Prefer odd k for binary classes; choose p (e.g., p=2) and small $\varepsilon > 0$ for stability.

From Baseline to KD-tree: Why Index for k-NN?

- **Extension idea:** Avoid scanning every point by *pruning entire regions at once*.
- Use a space-partitioning structure (KD-tree) to compute *lower bounds* on point distances and drive a **best-first** search.
- In low dimensions, this often yields **sublinear** queries; worst-case is still $O(n)$.

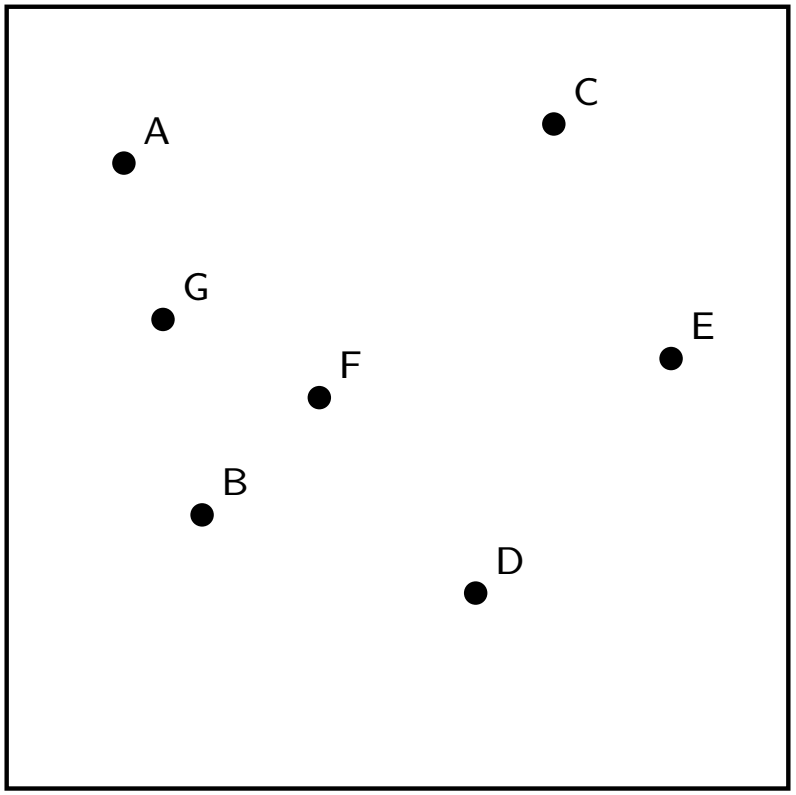
What is a KD-tree?

- A **K-D**imensional tree: a **binary space-partitioning** structure for $\mathbb{R}^d$.
- Each internal node stores a **split dimension** j and a **split value** s.
- The node's bounding box is cut into: `left` if $x_j \leq s$ and `right` if $x_j > s$.
- Leaves hold a small bucket of points (e.g., 1–32); every node keeps a **bounding box** that enables pruning.

Building a KD-tree

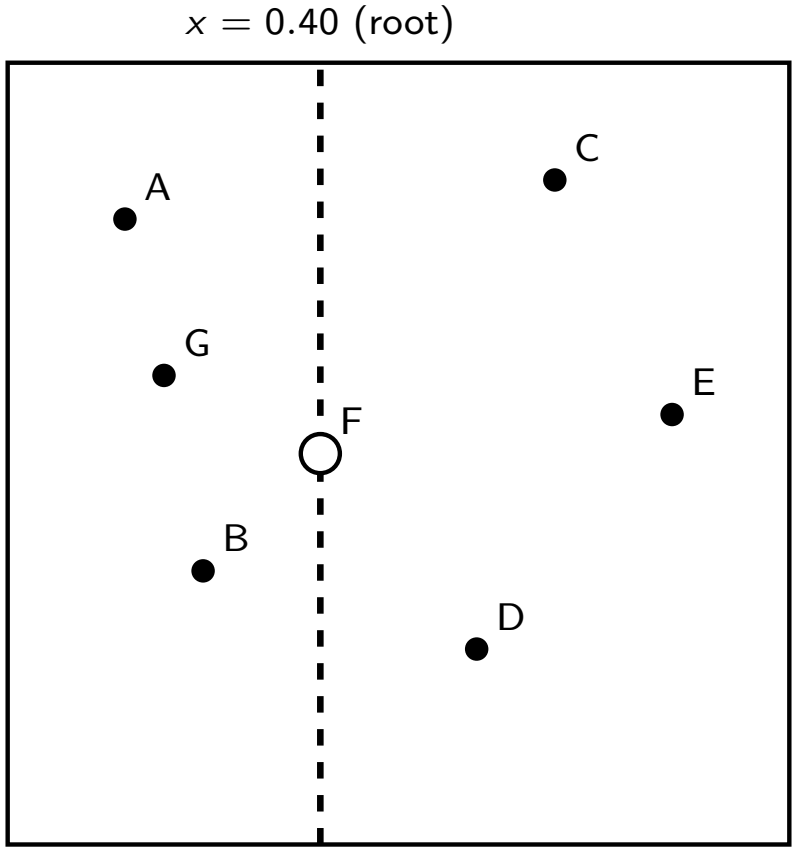
- **Input**: $\mathcal{X} \subset \mathbb{R}^d$, leaf size b.
- Split dimension: round-robin or **max variance/range**.
- Split value: typically the **median** ($O(n)$ selection or $O(n \log n)$ via sorting).
- **Complexity**: expected $O(n \log n)$ build, memory $O(n)$. Stop when node size $\leq b$ or depth limit.

KD-tree Construction (Animated)



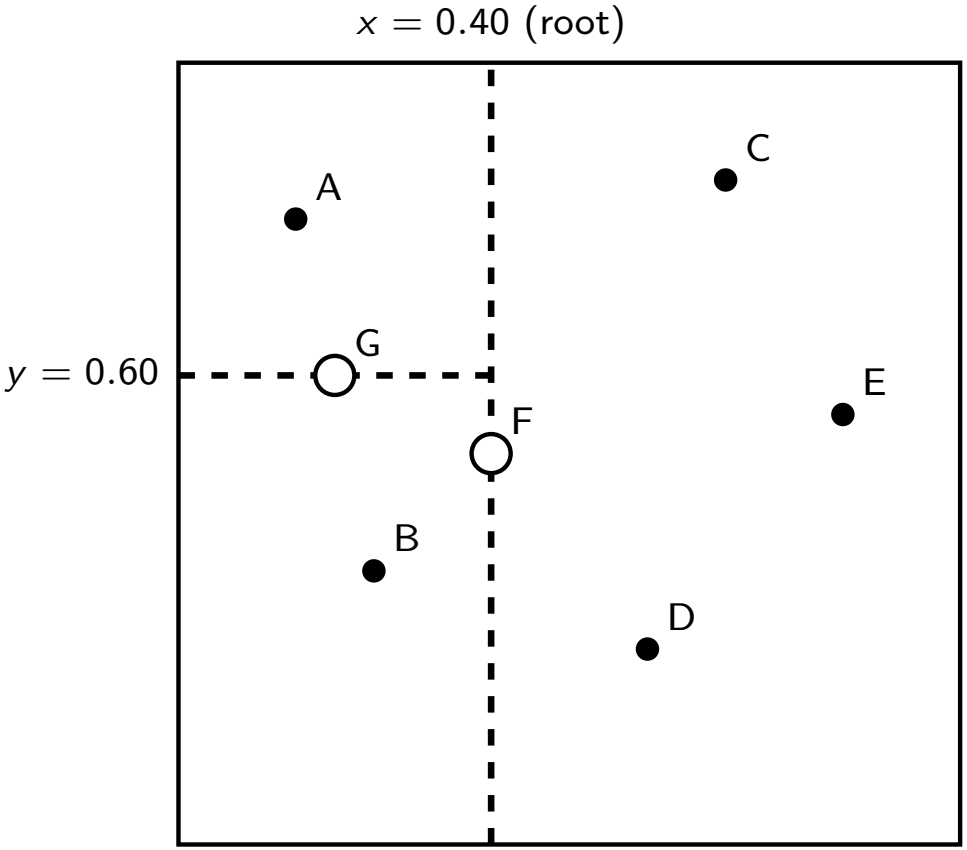
Step 1. Start with all points; the outer box shows the search space.

KD-tree Construction (Animated)



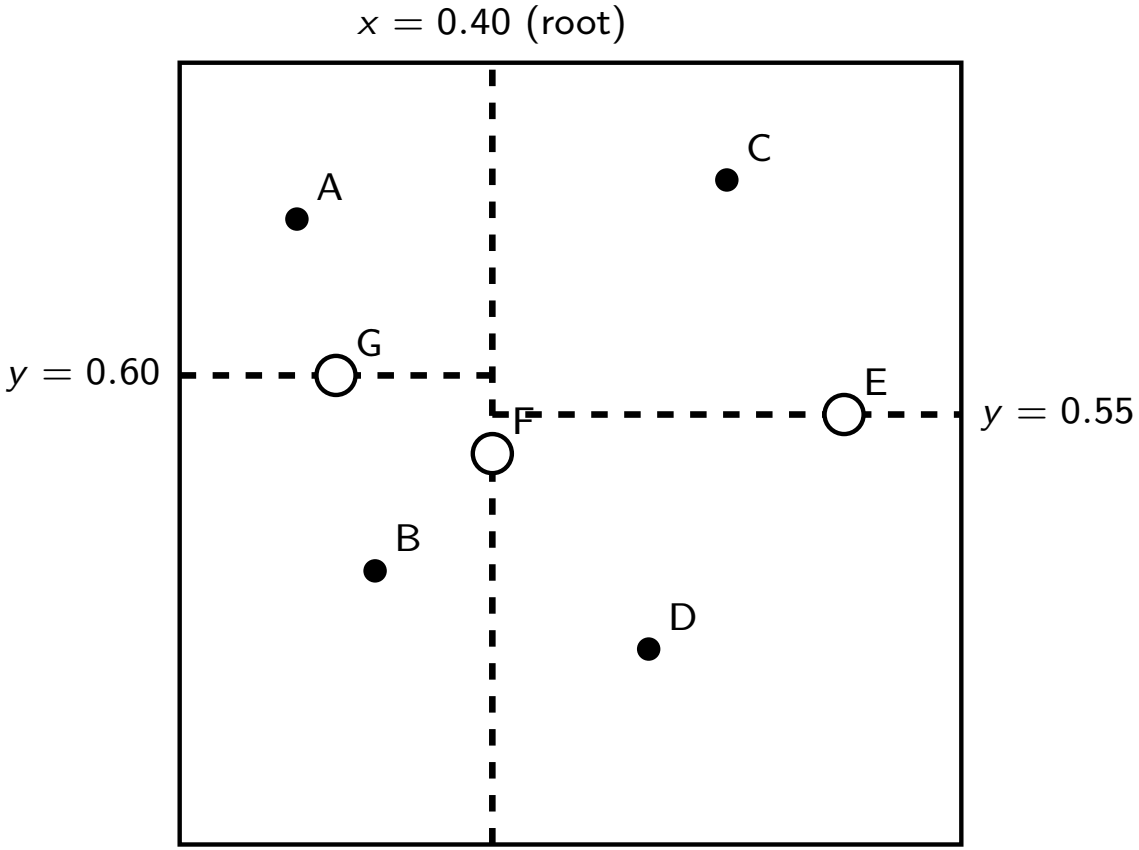
Step 2. Split by x at the median ($x = 0.40$). Highlight pivot F.

KD-tree Construction (Animated)



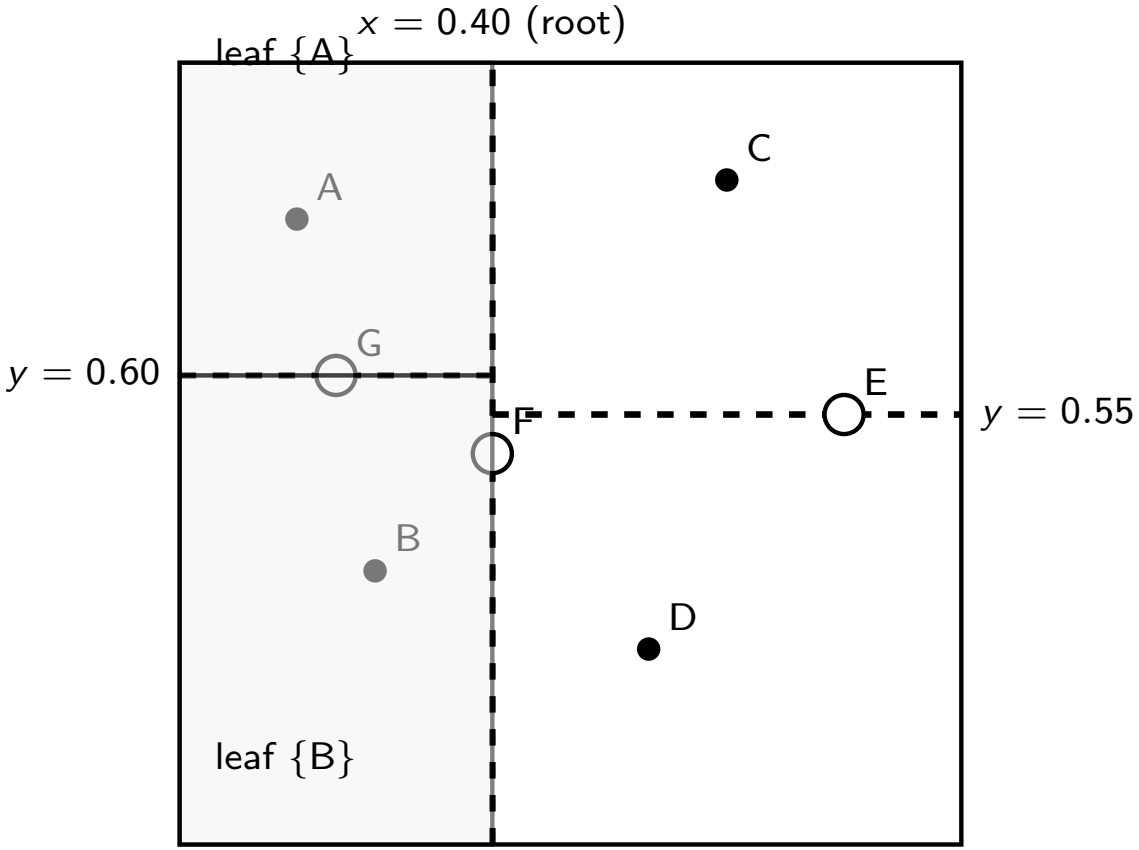
Step 3. Recurse on the left region $[0, 0.40] \times [0, 1]$: split by y at 0.60. Highlight G.

KD-tree Construction (Animated)



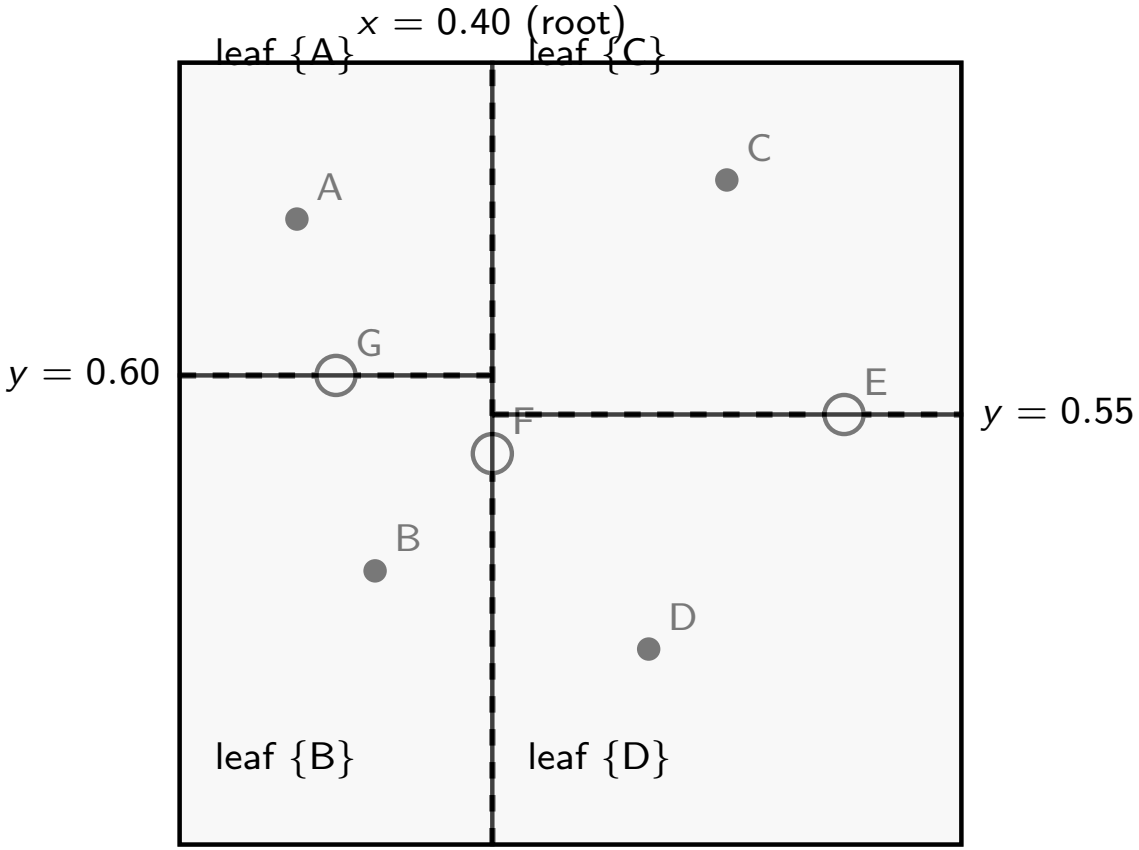
Step 4. Recurse on the right region $[0.40, 1] \times [0, 1]$: split by y at 0.55. Highlight E.

KD-tree Construction (Animated)



Step 5. Left subtree leaves: {A}, {B}.

KD-tree Construction (Animated)



Step 6. Right subtree leaves: {C}, {D}.

KD-tree Example

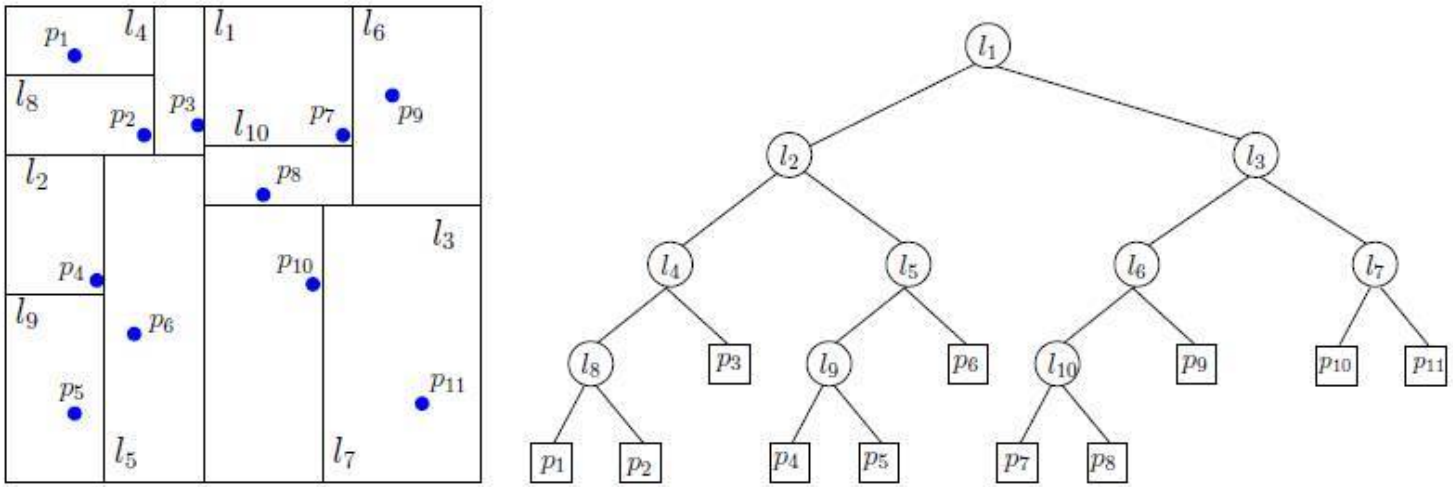


Figure: 2D KD-tree partitioning example

Geometry: Distance to an Axis-Aligned Box

Let $\mathrm{Box} = [l_1, u_1] \times \cdots \times [l_d, u_d]$ and $\boldsymbol{q} = (q_1, \ldots, q_d)$. The squared Euclidean distance is

$$\mathrm{dist}^2(\boldsymbol{q}, \mathrm{Box}) = \sum_{j=1}^{d} \begin{cases} (q_j - u_j)^2, & q_j > u_j, \\ (l_j - q_j)^2, & q_j < l_j, \\ 0, & \text{otherwise.} \end{cases}$$

If $\mathrm{dist}(\boldsymbol{q}, \mathrm{Box})$ exceeds the current radius τ, no point inside that box can improve the answer.

Intuition: Single-Queue k-NN

- **Key idea.** Run a *best-first* search driven by one min-heap (PQ) that holds *both* KD-tree nodes and data points. The PQ key is:

$$\text{key} = \begin{cases} \mathrm{dist}(\mathbf{q}, \text{Box}(\text{node})) & \text{(a valid lower bound)}, \\ \mathrm{dist}(\mathbf{q}, \mathbf{p}) & \text{(exact for a point)}. \end{cases}$$

- **Invariant (monotone keys).** Every time we pop from PQ, we remove the *smallest available* key. Because box distances are lower bounds, no unseen point can have distance smaller than the smallest key still in PQ.
- **Why correctness holds.** When a `point` is popped, its key equals its exact distance and is *no larger* than any remaining key (node lower bounds or other point distances). Hence it is the next true nearest. Repeating this k times yields the exact k-NN in increasing order.
- **Operational picture.**
  1. Start by pushing the root node with its box lower bound.
  2. Always expand the *most promising* entry: if it's a node, push its children with their box bounds; if it's a leaf, push its points with exact distances.
  3. If it's a point, *emit* it as the next neighbor.

k-NN with KD-tree (Single Priority Queue)

```
# Each PQ entry is (key, kind, payload) where:
#   kind  {"node","point"}
#   key  = dist(q, box)  if kind=="node"
#        = dist(q, p)    if kind=="point"

def kd_knn_single_queue(root, q, k):
    PQ = MinHeap()
    PQ.push((0.0, "node", root))
    result = []                        # will contain (dist, point) in ascending order

    while PQ and len(result) < k:
        key, kind, obj = PQ.pop_min()

        if kind == "point":
            result.append((key, obj))     # 'key' is exact dist(q, obj)
            continue

        node = obj
        if node.is_leaf:
            # Push all leaf points with *exact* distances
            for p in node.points:
                d = dist(q, p)            # squared distance is fine
                PQ.push((d, "point", p))
        else:
            # Expand children with lower-bound keys
            for child in (node.left, node.right):
                lb = box_distance(q, child.box)  # valid lower bound
                PQ.push((lb, "node", child))

    return result                          # already in nearest-first order
```

Notes. Uses only one min-heap. No separate neighbor heap or explicit radius τ is required: points are popped in increasing distance order.

k-NN via Best-First Search (Two Priority Queues)

- **Maintain two heaps:**
  - **PQ_nodes** (min-heap): key $= \mathrm{dist}(\boldsymbol{q}, \text{Box})$ (a lower bound).
  - **PQ_nbrs** (size-k max-heap): key = point distance; top holds current radius τ.
- Initialize with the root in PQ_nodes and $\tau = +\infty$. Pop the node with smallest bound; if leaf, evaluate points; if internal, push children with their bounds.
- **Stop when** PQ_nodes.min_key $\geq \tau$.

Pseudocode: k-NN with KD-tree (Two Priority Queues)

```
# KDNode: {is_leaf, left, right, box, points_or_None}

def kd_knn(root, q, k):
    PQ_nodes = MinHeap()              # key = dist(q, node.box)
    PQ_nbrs  = MaxHeap(capacity=k)    # key = dist(q, point)
    PQ_nodes.push((0.0, root))
    tau = float('inf')

    while PQ_nodes and PQ_nodes.min_key() < tau:
        _, node = PQ_nodes.pop_min()
        if node.is_leaf:
            for p in node.points:
                d = dist(q, p)
                if len(PQ_nbrs) < k:
                    PQ_nbrs.push((d, p))
                    tau = PQ_nbrs.max_key()
                elif d < PQ_nbrs.max_key():
                    PQ_nbrs.replace_max((d, p))
                    tau = PQ_nbrs.max_key()
        else:
            for child in (node.left, node.right):
                lb = box_distance(q, child.box)
                if lb < tau:
                    PQ_nodes.push((lb, child))
    return sorted(PQ_nbrs.items(), key=lambda t: t[0])
```

Complexity Summary

- **Baseline (no index)**: $\boxed{O(n \log k)}$ heap ops + $O(nd)$ distance work.
- **KD-tree Build**: expected $O(n \log n)$; memory $O(n)$.
- **KD-tree Query (low d)**: often $\approx O(\log n + k \log k)$ node visits; worst case $O(n)$.
- **Curse of dimensionality**: as d increases, box lower bounds get looser $\Rightarrow$ approaches linear scan.

Exact vs. Approximate k-NN

Exact

- Backtracking / best-first with tight bounds.
- Guarantees true k-NN.
- Can be slower in high d or adversarial layouts.

Approximate

- Early stop when PQ_nodes.min_key $\geq \tau(1+\epsilon)$.
- Or cap node/point visits or heap pops.
- Large speedups with small accuracy loss (tune ϵ).

Edge Cases and Robustness

- Duplicate points / ties: stable tie-breaking.
- Missing values: impute or drop features; KD-tree assumes total order per split dimension.
- Non-Euclidean metrics: ensure a valid lower bound for box distance or switch to metric trees.
- Dynamic data: KD-tree is static; for many updates, consider periodic rebuilds or dynamic variants.

Takeaways

- **First**, set a strong exact baseline with the *no-index* $O(n \log k)$ method.
- **Then**, extend to KD-tree with lower-bound-driven *best-first* search to achieve **sublinear** queries in low dimensions.
- For high-dimensional or web-scale settings, consider **approximate** indexes (e.g., HNSW, DiskANN) or alternative trees.